

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**

# **ĐỒ ÁN TỐT NGHIỆP**

**Xây dựng nền tảng thương mại điện tử đa thuê bao  
với kiến trúc Zero-file và Dynamic Rendering**

**Trịnh Vi Giang Xuân**

xuan.tvg225118@sis.hust.edu.vn

**Chương trình đào tạo: Công nghệ thông tin**

**Giảng viên hướng dẫn:** TS. Nguyễn Quốc Tuấn

\_\_\_\_\_

Chữ kí GVHD

**Khoa:** Khoa học máy tính

**Trường:** Công nghệ Thông tin và Truyền thông

**HÀ NỘI, 06/2022**

# LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành nhất đến TS. Nguyễn Quốc Tuấn. Thầy đã tận tình hướng dẫn, định hướng phát triển và chỉ bảo cặn kẽ, giúp em hoàn thành đồ án tốt nghiệp này.

Em cũng xin trân trọng cảm ơn Ban Giám hiệu Trường Công nghệ Thông tin và Truyền thông cùng quý thầy cô trong trường. Những kiến thức nền tảng quý giá mà các thầy cô truyền đạt chính là cơ sở vững chắc để em thực hiện đề tài. Bên cạnh đó, em xin tri ân gia đình, bạn bè đã luôn động viên, hỗ trợ và tạo mọi điều kiện thuận lợi cho em trong suốt quá trình học tập.

Cuối cùng, em xin cam đoan các kết quả trong báo cáo là thành quả trung thực của bản thân. Dù đã nỗ lực, đồ án vẫn khó tránh khỏi thiếu sót do giới hạn về kiến thức và kinh nghiệm. Em rất mong nhận được sự góp ý từ quý thầy cô để đề tài được hoàn thiện hơn.

# TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh thương mại điện tử tại Việt Nam đang phát triển mạnh mẽ, các doanh nghiệp vừa và nhỏ thường xuyên đối mặt với rào cản lớn về mặt chi phí cũng như rào cản kỹ thuật khi muốn xây dựng và vận hành một cửa hàng trực tuyến độc lập. Các nền tảng thương mại điện tử hiện có trên thị trường như Shopify, WooCommerce hay Haravan vẫn tồn tại nhiều hạn chế đáng kể, đặc biệt là trong việc không thể đáp ứng đồng thời cả ba tiêu chí: chi phí khởi tạo và duy trì thấp, khả năng tùy biến giao diện linh hoạt mà không yêu cầu kiến thức lập trình, và khả năng mở rộng hệ thống để phục vụ hàng chục ngàn cửa hàng mà không làm gia tăng tuyến tính chi phí hạ tầng. Xuất phát từ thực tiễn đó, đồ án này tập trung vào việc nghiên cứu và phát triển nền tảng thương mại điện tử đa thuê bao mang tên ShopVolo v2 nhằm giải quyết triệt để bài toán tối ưu hóa nguồn lực. Hướng tiếp cận chủ đạo của nghiên cứu là áp dụng kiến trúc phần mềm Multi-tenant SaaS kết hợp với nguyên lý thiết kế "Zero-file", trong đó toàn bộ giao diện của các cửa hàng được biểu diễn dưới dạng dữ liệu cấu hình thay vì mã nguồn vật lý riêng biệt.

Hệ thống được phát triển dựa trên mô hình Monorepo với công cụ Turborepo, tích hợp các nền tảng công nghệ hiện đại bao gồm Next.js cho giao diện trang quản trị và kết xuất mặt tiền cửa hàng động, NestJS cho hệ thống API lõi với khả năng cô lập dữ liệu người dùng tự động, cùng hệ thống cơ sở dữ liệu kép kết hợp giữa PostgreSQL và MongoDB để xử lý hiệu quả cả thông tin có cấu trúc lẫn tài liệu cấu hình linh hoạt. Điểm nhấn của kiến trúc hệ thống là thuật toán Deep Merge [1], cho phép trộn lẫn cấu trúc giao diện chuẩn và cấu hình tùy chỉnh của từng cá nhân theo thời gian thực, kết hợp với bộ nhớ đệm Redis để đảm bảo tốc độ phản hồi gần như tức thời. [TODO: Bổ sung chi tiết về những đóng góp chính và các kết quả hiệu năng cụ thể đạt được sau quá trình triển khai, kiểm thử hệ thống].

Sinh viên thực hiện  
(Ký và ghi rõ họ tên)

# ABSTRACT

In the context of the rapidly growing e-commerce sector in Vietnam, small and medium-sized enterprises (SMEs) frequently encounter significant financial and technical barriers when attempting to build and operate independent online stores. Existing e-commerce platforms on the market, such as Shopify, WooCommerce, or Haravan, still exhibit considerable limitations. Specifically, they fail to simultaneously satisfy three crucial criteria: low initial and maintenance costs, flexible interface customization without requiring programming knowledge, and system scalability capable of serving tens of thousands of stores without linearly increasing infrastructure costs. Stemming from this practical need, this thesis focuses on the research and development of a multi-tenant e-commerce platform named ShopVolo v2 to thoroughly address the resource optimization problem. The primary approach of the study involves implementing a Multi-tenant Software as a Service (SaaS) architecture combined with the "Zero-file" design principle, wherein the entire storefront interface is represented as configuration data rather than discrete physical source code files.

The system is developed based on a Monorepo model using the Turborepo tool, integrating modern technology stacks including Next.js for the administrative dashboard interface and dynamic storefront rendering, and NestJS for the core API system with automatic user data isolation capabilities. Furthermore, a dual database system combining PostgreSQL and MongoDB is employed to efficiently process both structured information and flexible configuration documents. The highlight of the system architecture is the Deep Merge algorithm, which allows the real-time blending of the standard interface structure with individual custom configurations, combined with Redis caching to ensure near-instantaneous response speeds. [TODO: Supplement details regarding the main contributions and specific performance results achieved after the deployment and testing of the system].

## MỤC LỤC

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....</b>                            | <b>1</b>  |
| 1.1 Đặt vấn đề.....                                                | 1         |
| 1.2 Mục tiêu và phạm vi đề tài.....                                | 2         |
| 1.3 Định hướng giải pháp.....                                      | 3         |
| 1.4 Bố cục đồ án .....                                             | 4         |
| <b>CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....</b>                | <b>6</b>  |
| 2.1 Khảo sát hiện trạng .....                                      | 6         |
| 2.2 Tổng quan chức năng .....                                      | 7         |
| 2.2.1 Biểu đồ use case tổng quát .....                             | 8         |
| 2.2.2 Biểu đồ use case phân rã — Quản lý Shop.....                 | 8         |
| 2.2.3 Biểu đồ use case phân rã — Quản lý Sản phẩm & Đơn hàng ..... | 9         |
| 2.2.4 Biểu đồ use case phân rã — Trải nghiệm Khách hàng.....       | 9         |
| 2.2.5 Quy trình nghiệp vụ — Onboarding và Dynamic Rendering .....  | 10        |
| 2.3 Đặc tả chức năng .....                                         | 10        |
| 2.3.1 Đặc tả use case: Khởi tạo Shop (UC-01).....                  | 11        |
| 2.3.2 Đặc tả use case: Tùy biến giao diện (UC-03).....             | 12        |
| 2.3.3 Đặc tả use case: Quản lý sản phẩm (UC-04) .....              | 13        |
| 2.3.4 Đặc tả use case: Duyệt sản phẩm động (UC-07).....            | 15        |
| 2.3.5 Đặc tả use case: Thanh toán và Đơn hàng (UC-09).....         | 16        |
| 2.4 Yêu cầu phi chức năng .....                                    | 17        |
| <b>CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....</b>                            | <b>19</b> |
| 3.1 Turborepo — Quản lý Monorepo .....                             | 19        |
| 3.2 NestJS — Framework Backend .....                               | 19        |
| 3.3 Next.js — Framework Frontend .....                             | 20        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 3.4 PostgreSQL và Prisma ORM — Cơ sở dữ liệu quan hệ .....        | 20        |
| 3.5 MongoDB và Mongoose — Cơ sở dữ liệu tài liệu .....            | 20        |
| 3.6 Redis — Cache In-memory .....                                 | 21        |
| 3.7 MinIO — Object Storage.....                                   | 21        |
| 3.8 Better Auth — Xác thực và Phân quyền .....                    | 22        |
| 3.9 Docker và Kubernetes — Containerization và Orchestration..... | 22        |
| <b>CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG ....</b>   | <b>25</b> |
| 4.1 Thiết kế kiến trúc.....                                       | 25        |
| 4.1.1 Lựa chọn kiến trúc phần mềm .....                           | 25        |
| 4.1.2 Thiết kế tổng quan.....                                     | 25        |
| 4.1.3 Thiết kế chi tiết gói .....                                 | 26        |
| 4.2 Thiết kế chi tiết.....                                        | 27        |
| 4.2.1 Thiết kế giao diện .....                                    | 27        |
| 4.2.2 Thiết kế lớp .....                                          | 27        |
| 4.2.3 Thiết kế cơ sở dữ liệu .....                                | 29        |
| 4.3 Xây dựng ứng dụng.....                                        | 31        |
| 4.3.1 Thư viện và công cụ sử dụng.....                            | 31        |
| 4.3.2 Kết quả đạt được .....                                      | 31        |
| 4.3.3 Minh họa các chức năng chính .....                          | 31        |
| 4.4 Kiểm thử.....                                                 | 32        |
| 4.5 Triển khai .....                                              | 32        |

## DANH MỤC HÌNH VẼ

|          |                                                                              |    |
|----------|------------------------------------------------------------------------------|----|
| Hình 2.1 | Biểu đồ use case tổng quát hệ thống ShopVolo v2 . . . . .                    | 8  |
| Hình 2.2 | Biểu đồ use case phân rã nhóm Quản lý Shop . . . . .                         | 8  |
| Hình 2.3 | Biểu đồ use case phân rã nhóm Sản phẩm và Đơn hàng . . . . .                 | 9  |
| Hình 2.4 | Biểu đồ use case phân rã nhóm trải nghiệm Khách hàng . . . . .               | 9  |
| Hình 2.5 | Biểu đồ hoạt động quy trình Onboarding và Rendering động .                   | 10 |
| Hình 2.6 | Biểu đồ hoạt động luồng tải lên và xử lý ảnh sản phẩm . . . . .              | 14 |
| Hình 2.7 | Biểu đồ hoạt động quy trình thanh toán và xác nhận đơn hàng                  | 17 |
|          |                                                                              |    |
| Hình 4.1 | Thiết kế chi tiết gói trong <code>api-core</code> : Luồng xử lý đa hệ thuê   | 26 |
| Hình 4.2 | Biểu đồ thiết kế chi tiết các lớp cốt lõi . . . . .                          | 28 |
| Hình 4.3 | Biểu đồ trình tự: Luồng Checkout (Đặt hàng) . . . . .                        | 28 |
| Hình 4.4 | Biểu đồ trình tự: Luồng xác thực BetterAuthGuard . . . . .                   | 29 |
| Hình 4.5 | Sơ đồ E-R các thực thể cốt lõi của hệ thống<br>(PostgreSQL/Prisma) . . . . . | 30 |

## DANH MỤC BẢNG BIỂU

|          |                                                             |    |
|----------|-------------------------------------------------------------|----|
| Bảng 1.1 | So sánh các nền tảng thương mại điện tử . . . . .           | 3  |
| Bảng 2.1 | So sánh chi tiết các nền tảng thương mại điện tử . . . . .  | 7  |
| Bảng 2.2 | Đặc tả use case Khởi tạo Shop (UC-01) . . . . .             | 11 |
| Bảng 2.3 | Đặc tả use case Tùy biến thương hiệu (UC-03) . . . . .      | 12 |
| Bảng 2.4 | Đặc tả use case Quản lý sản phẩm (UC-04) . . . . .          | 13 |
| Bảng 2.5 | Đặc tả use case Duyệt sản phẩm động (UC-07) . . . . .       | 15 |
| Bảng 2.6 | Đặc tả use case Thanh toán và Đơn hàng (UC-09) . . . . .    | 16 |
| Bảng 3.1 | Tổng hợp công nghệ sử dụng trong hệ thống ShopVolo v2 . .   | 23 |
| Bảng 4.1 | Danh sách thư viện và công cụ sử dụng trong dự án . . . . . | 31 |
| Bảng 4.2 | Thống kê thông số kỹ thuật dự án . . . . .                  | 31 |
| Bảng 4.3 | Các trường hợp kiểm thử chức năng Xác thực . . . . .        | 32 |



## DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

| Thuật ngữ | Ý nghĩa                                                                                        |
|-----------|------------------------------------------------------------------------------------------------|
| ACID      | Tính nguyên tử, nhất quán, độc lập và bền vững (Atomicity, Consistency, Isolation, Durability) |
| API       | Giao diện lập trình ứng dụng (Application Programming Interface)                               |
| CDN       | Mạng phân phối nội dung (Content Delivery Network)                                             |
| CI/CD     | Tích hợp liên tục và Triển khai liên tục (Continuous Integration/Continuous Deployment)        |
| CRUD      | Tạo, Đọc, Cập nhật, Xóa (Create, Read, Update, Delete)                                         |
| DAG       | Đồ thị có hướng phi chu trình (Directed Acyclic Graph)                                         |
| DDL       | Ngôn ngữ định nghĩa dữ liệu (Data Definition Language)                                         |
| ER        | Thực thể - Kết hợp (Entity-Relationship)                                                       |
| EUD       | Phát triển ứng dụng người dùng cuối (End-User Development)                                     |
| GWT       | Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)                         |
| HTML      | Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)                                     |
| IaaS      | Dịch vụ hạ tầng                                                                                |
| ISR       | Tái tạo tĩnh tăng dần (Incremental Static Regeneration)                                        |
| JSON      | JavaScript Object Notation                                                                     |
| JWT       | JSON Web Token                                                                                 |
| ODM       | Ánh xạ tài liệu đối tượng (Object-Document Mapping)                                            |

| Thuật ngữ | Ý nghĩa                                                            |
|-----------|--------------------------------------------------------------------|
| ORM       | Ánh xạ quan hệ đối tượng<br>(Object-Relational Mapping)            |
| RBAC      | Kiểm soát truy cập dựa trên vai trò<br>(Role-Based Access Control) |
| REST      | Representational State Transfer                                    |
| S3        | Dịch vụ lưu trữ đơn giản (Simple<br>Storage Service)               |
| SaaS      | Software as a Service — Phần mềm<br>dịch vụ                        |
| SME       | Doanh nghiệp vừa và nhỏ (Small and<br>Medium Enterprise)           |
| SSR       | Kết xuất phía máy chủ (Server-Side<br>Rendering)                   |
| TTL       | Thời gian sống (Time To Live)                                      |
| UC        | Trường hợp sử dụng (Use Case)                                      |
| UI        | Giao diện người dùng (User Interface)                              |
| UML       | Ngôn ngữ mô hình hóa thống nhất<br>(Unified Modeling Language)     |
| UX        | Trải nghiệm người dùng (User<br>Experience)                        |

## CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này giới thiệu tổng quan về bối cảnh và mục tiêu của đề tài nghiên cứu. Đầu tiên, phần 1.1 đặt vấn đề từ thực tiễn nhu cầu xây dựng cửa hàng trực tuyến của các doanh nghiệp vừa và nhỏ tại Việt Nam. Tiếp đó, phần 1.2 phân tích các giải pháp hiện có trên thị trường để làm rõ những hạn chế và xác định các mục tiêu cụ thể của hệ thống đề xuất. Phần 1.3 trình bày ngắn gọn định hướng kiến trúc đa thuê bao và công nghệ được lựa chọn để giải quyết bài toán. Cuối cùng, phần 1.4 mô tả bố cục chi tiết của toàn bộ báo cáo đồ án tốt nghiệp này.

### 1.1 Đặt vấn đề

Thương mại điện tử tại Việt Nam đang trải qua giai đoạn tăng trưởng mạnh mẽ trong những năm gần đây [2]. Sự chuyển dịch thói quen mua sắm của người tiêu dùng sang các nền tảng trực tuyến đã thúc đẩy các hộ kinh doanh và doanh nghiệp vừa và nhỏ liên tục tìm kiếm cơ hội mở rộng kênh bán hàng. Nhu cầu sở hữu một cửa hàng trực tuyến riêng biệt, chuyên nghiệp để xây dựng thương hiệu đang ngày càng trở nên cấp thiết.

Tuy nhiên, để xây dựng và vận hành một cửa hàng trực tuyến độc lập, các doanh nghiệp vừa và nhỏ phải đối mặt với nhiều rào cản lớn. Chi phí ban đầu cho việc thiết kế và phát triển website thường dao động từ mười đến năm mươi triệu đồng. Bên cạnh đó, việc duy trì một đội ngũ kỹ thuật để quản lý máy chủ, bảo mật và cập nhật tính năng liên tục là một gánh nặng tài chính không nhỏ. Hơn nữa, thời gian triển khai từ khi lên ý tưởng đến khi ra mắt thường kéo dài từ một đến ba tháng, làm chậm khả năng phản ứng với thị trường.

Các nền tảng phần mềm dưới dạng dịch vụ hiện có trên thị trường vẫn tồn tại nhiều hạn chế đáng kể. Các nền tảng quốc tế thường có chi phí duy trì hàng tháng cao, không tối ưu cho các phương thức thanh toán địa phương và giới hạn khả năng can thiệp sâu. Các giải pháp tự triển khai lại đòi hỏi người sử dụng phải có kiến thức kỹ thuật chuyên sâu để tự thiết lập hạ tầng và bảo mật. Trong khi đó, các nền tảng trong nước tuy dễ tiếp cận nhưng lại thiếu đi khả năng cho phép người dùng tự do tùy biến giao diện một cách sâu rộng mà không cần viết mã. Hiện tại, chưa có giải pháp nào đáp ứng đồng thời cả ba tiêu chí: chi phí thấp, khả năng tùy biến cao, và khả năng vận hành hàng chục ngàn cửa hàng mà không làm tăng tuyến tính chi phí hạ tầng.

Một nền tảng phần mềm mở, cho phép các doanh nghiệp vừa và nhỏ tại Việt Nam khởi động cửa hàng trực tuyến chỉ trong vài phút mà không cần kỹ năng lập

trình sẽ là một bước tiến quan trọng. Giải pháp này không chỉ giúp tiết kiệm chi phí và thời gian triển khai, mà còn cung cấp một giao diện chuyên nghiệp, góp phần đẩy nhanh quá trình số hóa thương mại.

## 1.2 Mục tiêu và phạm vi đề tài

Trên thị trường hiện nay, có nhiều nền tảng cung cấp giải pháp xây dựng cửa hàng trực tuyến với các đặc điểm khác nhau. Shopify là một trong những nền tảng cung cấp phần mềm dưới dạng dịch vụ hàng đầu, cung cấp giao diện đẹp mắt nhưng có chi phí duy trì khá cao từ 29\$ đến 299\$ mỗi tháng [3]. Hơn nữa, nền tảng này cũng giới hạn khả năng tùy biến sâu vào mã nguồn giao diện, và chưa thực sự phù hợp với các doanh nghiệp nhỏ tại Việt Nam về mặt chi phí và hỗ trợ thanh toán nội địa.

WooCommerce là một thành phần mở rộng mã nguồn mở dành cho hệ thống quản trị nội dung WordPress, cho phép tạo cửa hàng trực tuyến hoàn toàn miễn phí [4]. Mặc dù mang lại khả năng tùy biến sâu, giải pháp này lại yêu cầu người sử dụng phải tự thuê máy chủ lưu trữ, tự cấu hình bảo mật và thường xuyên thực hiện các bản cập nhật. Việc này đòi hỏi trình độ kỹ thuật cao và tiêu tốn nhiều thời gian vận hành hệ thống.

Tại Việt Nam, Haravan là một nền tảng phổ biến, thân thiện với người dùng trong nước và tích hợp tốt các dịch vụ vận chuyển, thanh toán nội địa [5]. Tuy nhiên, giải pháp này vẫn có những hạn chế nhất định trong việc tùy biến giao diện nếu người dùng không biết lập trình, đồng thời hệ thống chưa được xây dựng dựa trên kiến trúc đa thuê bao thực sự tối ưu cho việc cá nhân hóa giao diện động. Đối với các doanh nghiệp lớn, Magento là một lựa chọn mạnh mẽ, nhưng đi kèm với sự phức tạp và chi phí triển khai cực kỳ đắt đỏ [6].

| Tiêu chí                         | Shopify | WooCommerce | Haravan    | ShopVolo v2 |
|----------------------------------|---------|-------------|------------|-------------|
| Chi phí thấp                     | ×       | ✓           | Trung bình | ✓           |
| Không cần lập trình              | ✓       | ×           | ✓          | ✓           |
| Tùy biến giao diện sâu           | Hạn chế | ✓ (cần dev) | ×          | ✓           |
| Multi-tenant thực sự             | ✓       | ×           | ×          | ✓           |
| Phù hợp SME Việt Nam             | ×       | Khó         | ✓          | ✓           |
| Vận hành 20k+ shop / hạ tầng nhỏ | N/A     | ×           | N/A        | ✓           |

**Bảng 1.1:** So sánh các nền tảng thương mại điện tử

Bảng 1.1 tổng hợp sự khác biệt giữa các nền tảng thương mại điện tử hiện nay.

Có thể thấy, các nền tảng hiện có chưa giải quyết được đồng thời ba vấn đề cốt lõi: (i) chi phí triển khai và duy trì thấp, (ii) khả năng tùy biến giao diện sâu rộng mà không cần lập trình, và (iii) khả năng vận hành một số lượng lớn cửa hàng trên một hạ tầng phần cứng tối ưu.

Đề tài hướng đến xây dựng nền tảng ShopVolo v2 nhằm giải quyết các bài toán trên với các chức năng chính bao gồm: (i) khởi tạo cửa hàng trực tuyến trong vài phút thông qua các bước hướng dẫn tự động, (ii) cung cấp công cụ tùy biến giao diện trực quan không cần lập trình, (iii) quản lý vòng đời sản phẩm và trạng thái đơn hàng đầy đủ, (iv) xử lý thanh toán trực tuyến qua các cổng thanh toán hoặc nhận hàng trả tiền, và (v) hỗ trợ ánh xạ tên miền tùy chỉnh riêng biệt cho từng cửa hàng.

### 1.3 Định hướng giải pháp

Để giải quyết bài toán vận hành số lượng lớn cửa hàng với chi phí hạ tầng tối ưu, đề tài lựa chọn kiến trúc phần mềm Multi-tenant SaaS kết hợp với nguyên lý thiết kế "Zero-file". Đây là phương pháp quản lý giao diện trong đó toàn bộ cấu trúc trang của các cửa hàng được biểu diễn hoàn toàn dưới dạng tệp dữ liệu cấu hình JSON thay vì các tệp mã nguồn riêng lẻ. Cách tiếp cận này cho phép một mã nguồn duy nhất có thể phục vụ hàng ngàn cửa hàng khác nhau mà không cần thực hiện quá trình triển khai lại từng ứng dụng riêng biệt.

Hệ thống ShopVolo v2 được xây dựng dưới dạng Monorepo sử dụng Turborepo, bao gồm bốn ứng dụng chính. Ứng dụng Admin Dashboard phát triển bằng Next.js 15 cho phép người bán quản lý cửa hàng toàn diện. Ứng dụng API Core viết bằng NestJS 11 cung cấp các giao diện lập trình ứng dụng REST API có khả năng cô lập dữ liệu tự động qua AsyncLocalStorage. Ứng dụng Storefront Engine cũng dùng Next.js 15 để kết xuất giao diện động từ cấu hình, cùng với một công cụ CLI Tool hỗ trợ vận hành hệ thống hàng loạt. Cơ sở dữ liệu PostgreSQL được dùng cho các dữ liệu có cấu trúc chặt chẽ như đơn hàng, trong khi MongoDB lưu trữ các dữ liệu có tính chất linh hoạt như cấu hình giao diện.

Đóng góp nổi bật của đề tài là thuật toán Deep Merge, kết hợp bộ khung giao diện Master Template mặc định với cấu hình riêng Tenant Config của từng cửa hàng. Cơ chế này chỉ lưu trữ những phần thay đổi, giúp tiết kiệm đáng kể không gian lưu trữ và kết hợp với chiến lược bộ nhớ đệm Redis có vòng đời một giờ nhằm đạt hiệu suất cao. Mục tiêu của hệ thống là có khả năng phục vụ đồng thời 20.000 cửa hàng trên một máy chủ 16GB RAM với thời gian phản hồi dưới 500ms đối với dữ liệu mới và dưới 10ms đối với dữ liệu đã lưu trong bộ nhớ đệm. Chi tiết về các giải pháp kỹ thuật này sẽ được trình bày cụ thể trong Chương 5.

## 1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày quá trình khảo sát hiện trạng từ ba nguồn chính bao gồm người dùng, hệ thống hiện có, và ứng dụng tương tự, qua đó xác định yêu cầu chức năng thông qua mười trường hợp sử dụng, đặc tả chi tiết năm trường hợp sử dụng quan trọng nhất, và đưa ra các yêu cầu phi chức năng về hiệu năng, bảo mật, cũng như hạ tầng hệ thống.

Chương 3 giới thiệu các công nghệ và nền tảng kiến trúc được lựa chọn để hiện thực hóa các yêu cầu đã xác định ở Chương 2, bao gồm Turborepo, NestJS, Next.js, Prisma, Mongoose, Redis, MinIO, và Docker, cùng lý do chọn mỗi công nghệ thay vì các giải pháp thay thế có sẵn trên thị trường.

Chương 4 trình bày thiết kế kiến trúc tổng quan theo mô hình phân tầng, thiết kế giao diện cho trang quản trị và trang hiển thị cửa hàng, thiết kế lớp cho các thành phần chính, lược đồ cơ sở dữ liệu PostgreSQL và MongoDB, cùng với kết quả xây dựng ứng dụng, quy trình kiểm thử và mô hình triển khai hệ thống.

Chương 5 phân tích chuyên sâu hai đóng góp kỹ thuật nổi bật của đề tài là cơ chế giao diện "Zero-file" với bộ phân giải thành phần động và thuật toán Deep Merge trộn dữ liệu giữa Master Template và cấu hình tùy chỉnh, kèm theo các kết quả đo lường và đánh giá hiệu năng thực tế.

Chương 6 tổng kết các kết quả đã đạt được trong quá trình nghiên cứu và phát triển, so sánh sản phẩm hoàn thiện với các nền tảng tương tự trên thị trường, đánh giá một cách khách quan những điểm chưa hoàn thiện và đề xuất các hướng phát triển tiếp theo.

Chương này đã làm rõ bài toán thực tiễn về sự thiếu hụt một nền tảng phù hợp cho các doanh nghiệp vừa và nhỏ, đồng thời phân tích những điểm còn hạn chế của các hệ thống thương mại điện tử hiện tại. Dựa trên cơ sở đó, đề tài đề xuất việc phát triển hệ thống ShopVolo v2 ứng dụng kiến trúc đa thuê bao và nguyên lý giao diện "Zero-file" để tối ưu hóa quá trình vận hành và tùy biến. Chương tiếp theo sẽ trình bày chi tiết kết quả khảo sát hiện trạng và phân tích yêu cầu của hệ thống.

## CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã xác định bài toán về rào cản chi phí và kỹ thuật mà các doanh nghiệp vừa và nhỏ gặp phải khi tiếp cận thương mại điện tử, đồng thời đề xuất giải pháp nền tảng ShopVolo v2. Chương 2 này sẽ đi sâu vào việc khảo sát và phân tích yêu cầu của hệ thống để làm cơ sở cho quá trình thiết kế. Cụ thể, phần 2.1 trình bày kết quả khảo sát hiện trạng từ ba nguồn chính nhằm nhận diện rõ vấn đề. Tiếp đó, phần 2.2 cung cấp cái nhìn tổng quan về các chức năng của hệ thống thông qua các biểu đồ phân rã. Phần 2.3 đi vào đặc tả chi tiết luồng nghiệp vụ của các trường hợp sử dụng cốt lõi. Cuối cùng, phần 2.4 đưa ra các yêu cầu phi chức năng cần đáp ứng để hệ thống vận hành trơn tru.

### 2.1 Khảo sát hiện trạng

Đối tượng người dùng của ShopVolo v2 là các doanh nghiệp vừa và nhỏ tại Việt Nam có nhu cầu mở rộng kênh bán hàng trực tuyến. Qua phân tích báo cáo về thương mại điện tử Việt Nam [7], hơn 60% các đơn vị doanh nghiệp vừa và nhỏ phản ánh chi phí xây dựng website đang là rào cản lớn nhất của họ. Người dùng đang rất cần một nền tảng thương mại điện tử thực sự dễ dùng mà không yêu cầu kiến thức viết mã, có mức chi phí thấp để duy trì, nhưng vẫn cung cấp được một giao diện chuyên nghiệp. Dựa vào những nhu cầu cấp thiết đó, hệ thống phân tích và chỉ ra ba nhóm người dùng chính bao gồm: (i) Super Admin giữ vai trò quản trị toàn bộ hệ thống nền tảng, (ii) Tenant Owner là những người kinh doanh trực tiếp quản lý cửa hàng của mình, và (iii) End Customer đại diện cho lượng khách hàng cuối cùng sẽ truy cập vào nền tảng để mua sắm.

Trước khi nền tảng ShopVolo v2 được định hình, các cửa hàng thường tự xây dựng trang web bằng các công cụ như WordPress kết hợp WooCommerce hoặc dựa hoàn toàn vào các trang mạng xã hội phổ biến như Facebook và Zalo. Tuy nhiên, các giải pháp này bộc lộ rất nhiều hạn chế cốt lõi như không có hệ thống quản lý đơn hàng tập trung, trải nghiệm giao diện không nhất quán trên các thiết bị, và gần như không thể mở rộng quy mô khi số lượng sản phẩm cũng như lượng truy cập tăng vọt.

Khi xét đến các ứng dụng phần mềm dịch vụ chuyên biệt tương tự hiện nay, Shopify nổi bật với tư cách là một nền tảng thương mại điện tử khổng lồ cung cấp giao diện bắt mắt. Điểm yếu của nền tảng này là chi phí duy trì cố định hằng tháng khá cao và khả năng tùy biến sâu bị hạn chế [3]. Trong khi đó, WooCommerce lại cung cấp tính tự do tuyệt đối nhờ mã nguồn mở nhưng người sử dụng bị ép buộc phải tự cấu hình máy chủ, vá lỗi bảo mật liên tục, gây hao tổn nguồn lực kỹ thuật

đáng kể [4].

Tại thị trường trong nước, Haravan đã xây dựng được một hệ sinh thái hỗ trợ thanh toán nội địa vô cùng tiện lợi cho các nhà bán lẻ [5]. Tuy nhiên, công cụ thiết kế của họ vẫn còn cứng nhắc, và bản thân hệ thống chưa ứng dụng triệt để kiến trúc đa thuê bao nhằm tiết kiệm tài nguyên kết xuất giao diện. Ngoài ra, phân khúc doanh nghiệp quy mô cực lớn thường chọn giải pháp Adobe Commerce với sự mạnh mẽ tuyệt đối nhưng lại đòi hỏi nguồn kinh phí khổng lồ để bắt đầu và duy trì bảo trì hệ thống [6].

| Tiêu chí so sánh              | Shopify       | WooCommerce              | Haravan       | ShopVolo v2 (đề xuất)     |
|-------------------------------|---------------|--------------------------|---------------|---------------------------|
| Mô hình triển khai            | SaaS          | Self-hosted              | SaaS          | SaaS                      |
| Chi phí hàng tháng            | \$29–\$299    | Miễn phí (hosting riêng) | 200k–3tr VND  | Tối ưu (multi-tenant)     |
| Tùy biến giao diện            | Theme cố định | Cần dev WordPress        | Hạn chế       | Visual builder, Zero-file |
| Multi-tenancy thực sự         | Có            | Không                    | Không         | Có (AsyncLocalStorage)    |
| Hỗ trợ thanh toán VN          | Hạn chế       | Plugin                   | Có            | Có (Stripe, COD, MoMo)    |
| Vận hành 20k+ shop / RAM thấp | Không áp dụng | Không                    | Không áp dụng | Mục tiêu thiết kế         |

**Bảng 2.1:** So sánh chi tiết các nền tảng thương mại điện tử

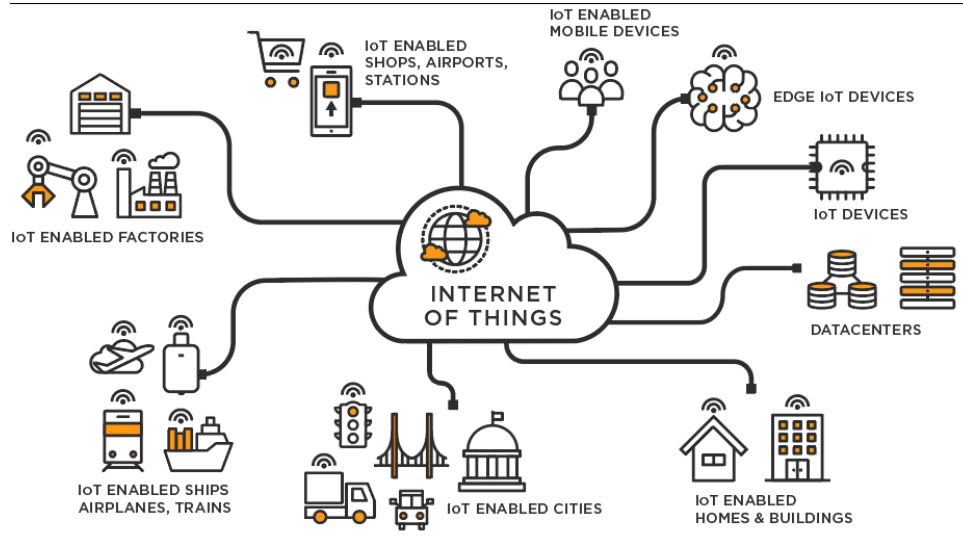
Bảng 2.1 cho thấy rõ ràng một sự thiếu hụt nghiêm trọng trên thị trường. Các hệ thống hiện tại đều chưa thể giải quyết triệt để vấn đề cân bằng giữa chi phí, khả năng vận hành nhiều cửa hàng trên cấu hình máy chủ hạn hẹp và công cụ thiết kế giao diện linh hoạt. Từ những hạn chế tổng quát kể trên, nền tảng của đề tài phải tập trung xây dựng 10 tính năng cốt lõi: (i) khởi tạo cửa hàng, (ii) quản lý bộ khung nền tảng, (iii) tùy biến thương hiệu, (iv) quản lý sản phẩm, (v) thiết lập menu điều hướng, (vi) tạo trang nội dung động, (vii) duyệt sản phẩm động trên trang cửa hàng, (viii) quản lý giỏ hàng, (ix) thanh toán và xử lý đơn hàng, và cuối cùng là (x) ánh xạ tên miền tùy chỉnh.

## 2.2 Tổng quan chức năng

Phần này có nhiệm vụ tóm tắt cấu trúc phân mảnh hệ thống thông qua các sơ đồ trường hợp sử dụng từ mức tổng thể đến mức chi tiết. Việc đặc tả chuyên sâu từng chức năng cốt lõi sẽ được tiến hành ở mục tiếp theo.



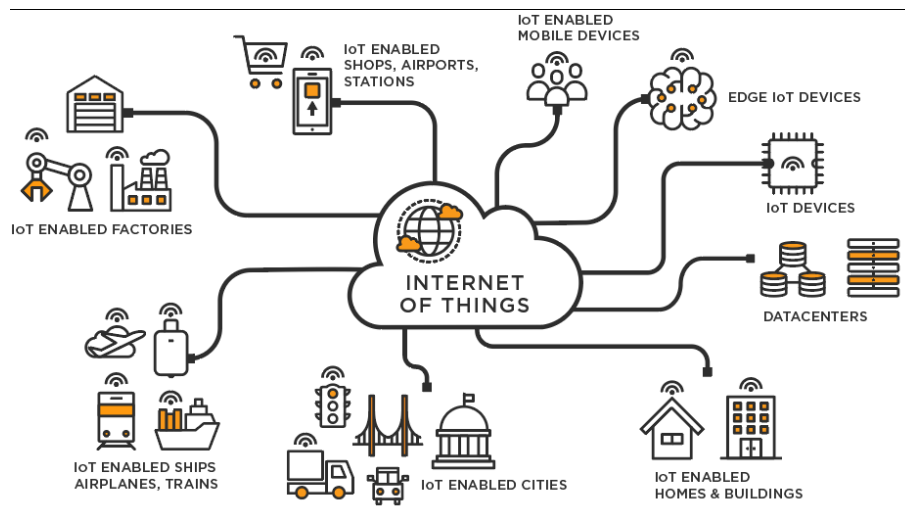
### 2.2.1 Biểu đồ use case tổng quát



**Hình 2.1:** Biểu đồ use case tổng quát hệ thống ShopVolo v2

Hình 2.1 mô tả cái nhìn toàn cảnh về mọi hành vi tương tác trong hệ thống. Tác nhân Super Admin là người nắm giữ quyền hành lớn nhất, chịu trách nhiệm quản lý toàn bộ nền tảng và thiết kế các mẫu bộ khung nền tảng cho cửa hàng. Tác nhân Tenant Owner là những người kinh doanh tham gia trực tiếp vào việc đăng ký, thiết lập, cấu hình và quản lý mọi góc ngách thông tin trong cửa hàng của riêng họ. Cuối cùng, tác nhân End Customer là tập hợp lượng khách hàng đồ sộ liên tục truy cập vào hệ thống mặt tiền hiển thị nhằm tìm kiếm sản phẩm và thanh toán.

### 2.2.2 Biểu đồ use case phân rã — Quản lý Shop

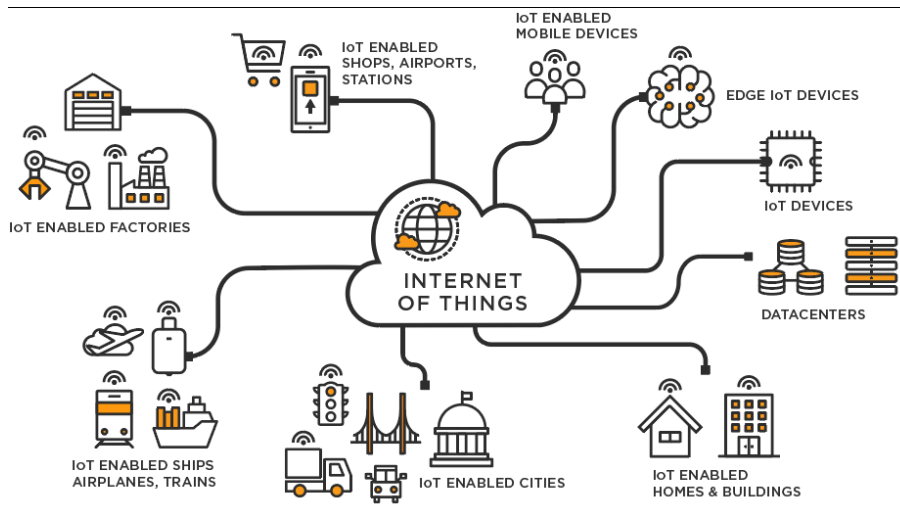


**Hình 2.2:** Biểu đồ use case phân rã nhóm Quản lý Shop

Hình 2.2 tập trung mô tả chi tiết các tác vụ liên quan đến việc thiết lập cửa hàng. Chức năng khởi tạo shop hỗ trợ đăng ký nhanh tài khoản và định danh cửa hàng.

Quản lý nền tảng mẫu cung cấp luồng tùy chỉnh giao diện gốc. Chức năng tùy biến thương hiệu giúp đổi mới hình ảnh mà không cần biết viết mã, còn thiết lập điều hướng và ánh xạ tên miền hỗ trợ tối đa cho việc định hình cấu trúc thông tin hoàn chỉnh.

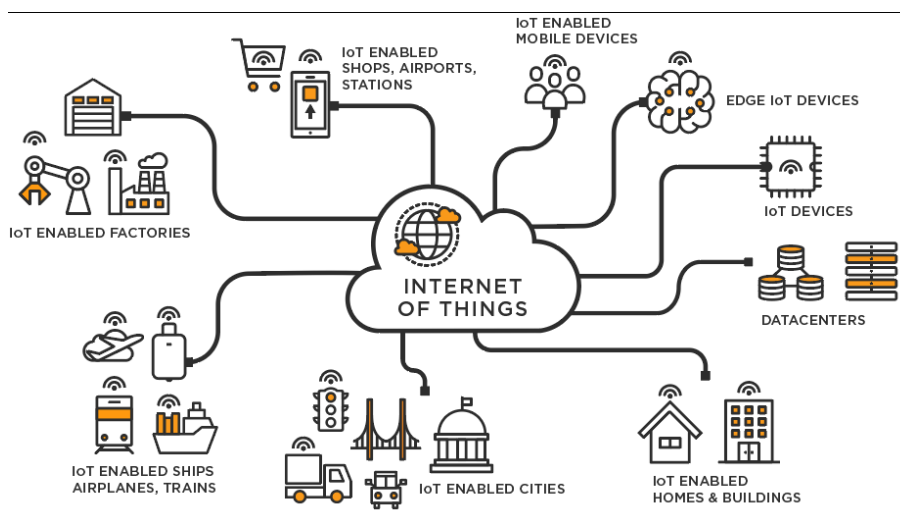
### 2.2.3 Biểu đồ use case phân rã — Quản lý Sản phẩm & Đơn hàng



**Hình 2.3:** Biểu đồ use case phân rã nhóm Sản phẩm và Đơn hàng

Hình 2.3 mô tả trực tiếp các chức năng sinh lời cốt lõi của một hệ thống bán lẻ. Quản lý sản phẩm cho phép người bán cập nhật liên tục mọi thay đổi về kho hàng và hình ảnh. Quản lý giỏ hàng đóng vai trò điểm neo kết nối mong muốn mua sắm của khách, còn chức năng thanh toán trực tiếp giải quyết dòng tiền giao dịch để hoàn tất đơn hàng một cách nhanh chóng.

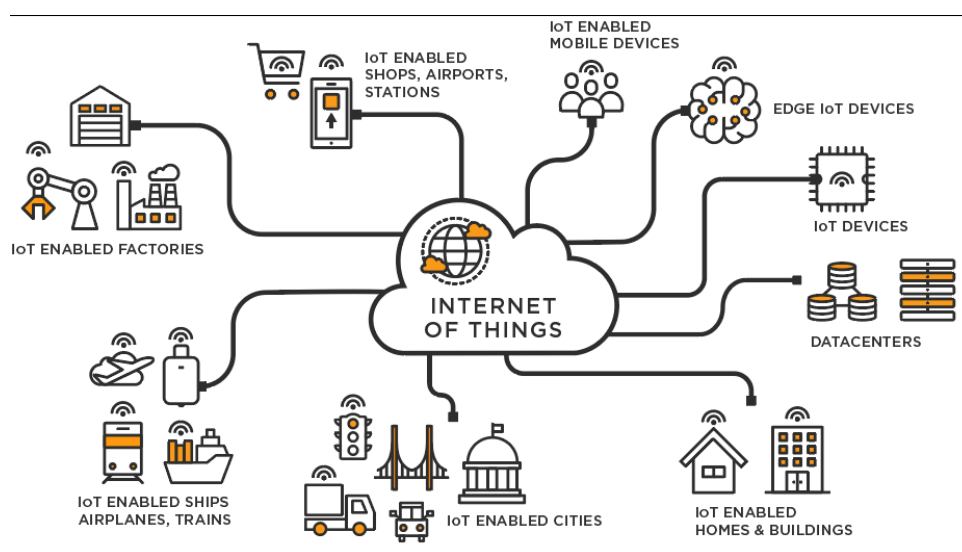
### 2.2.4 Biểu đồ use case phân rã — Trải nghiệm Khách hàng



**Hình 2.4:** Biểu đồ use case phân rã nhóm trải nghiệm Khách hàng

Hình 2.4 mô tả trải nghiệm người tiêu dùng trên bề mặt hệ thống. Khách hàng dễ dàng theo dõi thông tin cửa hàng qua các trang nội dung động, đồng thời liên tục tương tác với mặt tiền của cửa hàng thông qua chức năng duyệt sản phẩm được kết xuất mạnh mẽ từ cấu hình hệ thống.

### 2.2.5 Quy trình nghiệp vụ — Onboarding và Dynamic Rendering



**Hình 2.5:** Biểu đồ hoạt động quy trình Onboarding và Rendering động

Hình 2.5 mô tả quy trình nghiệp vụ quan trọng nhất, gắn kết vòng đời của cửa hàng từ khâu kiến tạo đến khâu hoạt động thực tế. Khi người dùng mới thực hiện đăng ký thông tin, hệ thống tự động sinh tài khoản, gán bộ khung giao diện tiêu chuẩn và cho phép cấu hình ngay lập tức. Cấu hình được xuất bản và mỗi khi có khách hàng tiến vào thông qua mạng lưới tên miền, máy chủ kết xuất lập tức kéo tệp dữ liệu về để tiến hành phép kết hợp đệ quy sâu. Toàn bộ các thông tin trộn lẫn này được hiển thị ngay lập tức lên màn hình một cách liền mạch nhất.

### 2.3 Đặc tả chức năng

Dựa trên mười tác vụ chính đã thống kê, phân hệ hệ thống này lựa chọn tiến hành đặc tả chuyên sâu năm quy trình vận hành trung tâm ảnh hưởng trực tiếp đến hiệu quả hoạt động tổng thể.

### 2.3.1 Đặc tả use case: Khởi tạo Shop (UC-01)

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tên use case</b>    | Khởi tạo Shop (Tenant Onboarding)                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Tác nhân</b>        | Tenant Owner                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Mục đích</b>        | Đăng ký tài khoản và tạo cửa hàng mới trên nền tảng                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Tiền điều kiện</b>  | Tenant Owner chưa có tài khoản; email và domain chưa tồn tại trong hệ thống                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Hậu điều kiện</b>   | Shop được tạo với trạng thái active; admin account được khởi tạo; Master Template mặc định được gán; email xác nhận được gửi                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Luồng chính</b>     | <p>1. Tenant Owner gửi POST /api/v1/tenants/register với: shopName, email, domain, ownerName</p> <p>2. Hệ thống kiểm tra email và domain chưa trùng lặp</p> <p>3. Hệ thống tạo bản ghi tenant trong PostgreSQL</p> <p>4. Hệ thống tạo user với role=OWNER, liên kết với tenant</p> <p>5. Hệ thống gán Master Template mặc định vào MongoDB (TenantLayout)</p> <p>6. Hệ thống khởi tạo navigation menu mặc định</p> <p>7. Hệ thống gửi email xác nhận</p> <p>8. API trả về tenantId, domain, status=active</p> |
| <b>Luồng phát sinh</b> | <p>[2a] Domain đã tồn tại → HTTP 409, code 1013, message "Domain already exists"</p> <p>[2b] Email đã đăng ký → HTTP 409, code 9996, message "Email already registered"</p> <p>[2c] Dữ liệu không hợp lệ → HTTP 400, code 1002/1003/1004</p>                                                                                                                                                                                                                                                                  |

**Bảng 2.2:** Đặc tả use case Khởi tạo Shop (UC-01)

Bảng 2.2 mô tả luồng đăng ký ban đầu thông qua một cấu trúc hàm tự động giúp thiết lập cơ sở dữ liệu phân tán ngay tức thì cho một cửa hàng mới.

### 2.3.2 Đặc tả use case: Tùy biến giao diện (UC-03)

|                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tên use case</b>                                                                                                                                                                                                                                                                                                                              | Tùy biến thương hiệu (Branding)                                                                                                            |
| <b>Tác nhân</b>                                                                                                                                                                                                                                                                                                                                  | Tenant Owner                                                                                                                               |
| <b>Mục đích</b>                                                                                                                                                                                                                                                                                                                                  | Thay đổi logo, màu sắc chủ đạo, và font chữ của cửa hàng                                                                                   |
| <b>Tiền điều kiện</b>                                                                                                                                                                                                                                                                                                                            | Tenant Owner đã đăng nhập; shop đang ở trạng thái active                                                                                   |
| <b>Hậu điều kiện</b>                                                                                                                                                                                                                                                                                                                             | Cấu hình branding được lưu vào MongoDB (TenantLayout.overrides); Redis cache bị invalidate; giao diện storefront cập nhật trong vòng 1 giờ |
| <b>Luồng chính</b><br><br>2. Dashboard gửi PATCH /api/v1/tenants/{tenantId}/config với branding JSON<br>3. API nhận override, thực hiện deep merge với Master Template<br>4. Lưu chỉ phần override (diff) vào MongoDB TenantLayout<br>5. Invalidate Redis cache key storefront:config:{tenantId}<br>6. Trả về config đã cập nhật và lastModified | 1. Tenant Owner chỉnh sửa logo/màu sắc/font trong Admin Dashboard                                                                          |
| <b>Luồng phát sinh</b><br><br>[3a] Màu sắc không đúng hex format → HTTP 400, message "Invalid color format"                                                                                                                                                                                                                                      | [3a] URL logo không hợp lệ → HTTP 400, message "Invalid logo URL"                                                                          |

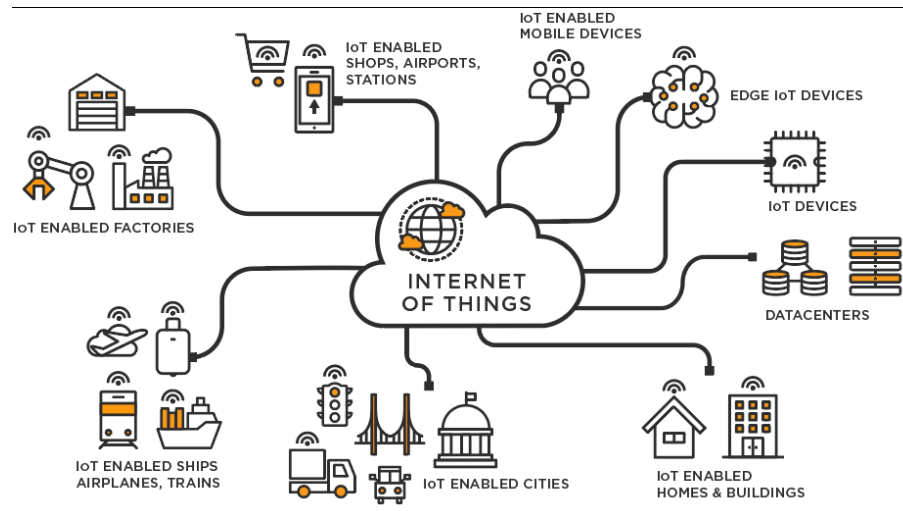
**Bảng 2.3:** Đặc tả use case Tùy biến thương hiệu (UC-03)

Bảng 2.3 mô tả quy trình tùy biến giúp lưu trữ lại đúng phần nội dung khác biệt thay vì toàn bộ mã nguồn, tối ưu hóa hệ thống dữ liệu đồ sộ.

### 2.3.3 Đặc tả use case: Quản lý sản phẩm (UC-04)

|                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Tên use case</b>                                                                                                                                                                                                                                                                                                                                                                                                                                      | Quản lý sản phẩm                                                                                                               |
| <b>Tác nhân</b>                                                                                                                                                                                                                                                                                                                                                                                                                                          | Tenant Owner                                                                                                                   |
| <b>Mục đích</b>                                                                                                                                                                                                                                                                                                                                                                                                                                          | Cho phép người quản lý thêm mới, chỉnh sửa, xóa bỏ sản phẩm và cập nhật hình ảnh                                               |
| <b>Tiền điều kiện</b>                                                                                                                                                                                                                                                                                                                                                                                                                                    | Chủ cửa hàng đã đăng nhập thành công vào phiên quản trị                                                                        |
| <b>Hậu điều kiện</b>                                                                                                                                                                                                                                                                                                                                                                                                                                     | Thông tin sản phẩm cập nhật lên máy chủ trung tâm; quy trình trí tuệ nhân tạo xử lý hình nền hoàn tất và lưu trữ vào kho MinIO |
| <b>Luồng chính</b><br><br>2. Admin Dashboard gửi yêu cầu cập nhật API hệ thống lõi<br>3. Hình ảnh thô đẩy trực tiếp lên kho chứa đối tượng MinIO<br>4. Trí tuệ nhân tạo nhận diện và bóc tách thông nền chuẩn bị qua CronJob máy chủ ảnh<br>5. Hình ảnh sau bóc tách được thu nhỏ theo ba mức độ phân giải tiêu chuẩn<br>6. Toàn bộ định danh ảnh thu nhỏ lưu ngược vào bộ nhớ MongoDB<br>7. Thông báo hoàn tất khởi tạo sản phẩm về màn hình người dùng | 1. Tenant Owner điền tên, mã SKU, và dữ liệu cấu thành sản phẩm                                                                |
| <b>Luồng phát sinh</b><br><br>[4a] Lỗi kỹ thuật máy chủ ảnh ngắt quãng → Hệ thống lưu cấu hình ảnh mặc định và kích hoạt lại sau                                                                                                                                                                                                                                                                                                                         | [1a] Trường dữ liệu định danh mã bị rỗng<br>→ Cảnh báo nhập liệu bắt buộc                                                      |

**Bảng 2.4:** Đặc tả use case Quản lý sản phẩm (UC-04)



**Hình 2.6:** Biểu đồ hoạt động luồng tải lên và xử lý ảnh sản phẩm

Hình 2.6 là quy trình quản lý thông tin sản phẩm và phân luồng làm sạch phòng nền ảnh thông minh giúp tăng độ sắc nét sản phẩm một cách chuyên nghiệp.

### 2.3.4 Đặc tả use case: Duyệt sản phẩm động (UC-07)

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tên use case</b>    | Duyệt sản phẩm động                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Tác nhân</b>        | End Customer                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Mục đích</b>        | Trình diễn giao diện nội dung sản phẩm mượt mà với khách mua hàng                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Tiền điều kiện</b>  | Tên miền cửa hàng đang trở đúng địa chỉ máy chủ và cửa hàng vẫn kích hoạt                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Hậu điều kiện</b>   | Toàn bộ nội dung thông tin được tải hoàn chỉnh và ghi nhận chỉ mục phục vụ truy xuất tức thời                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Luồng chính</b>     | <p>1. End Customer gõ tên miền truy cập vào ô tìm kiếm của trình duyệt mạng</p> <p>2. Lớp phần mềm trung gian tự động bóc tách chuỗi để xác định mã cửa hàng</p> <p>3. Động cơ mặt tiền kích hoạt hàm GET<br/>/api/v1/storefront/config?domain=xxx</p> <p>4. Máy chủ ứng dụng lập tức lỗi cấu hình đệ quy sâu từ kho lưu trữ nền</p> <p>5. Bản ghi nội dung trộn lẫn được đẩy thẳng vào vùng nhớ tạm Redis với tuổi thọ giới hạn một giờ</p> <p>6. Bộ phân giải thành phần cấu trúc JSON sang bộ giao diện lập trình bề mặt React</p> <p>7. Nội dung phản hồi được truyền tới màn hình phía khách truy cập hoàn hảo nhất</p> |
| <b>Luồng phát sinh</b> | <p>[2a] Mã cửa hàng bị khóa do hết hạn thanh toán → Hệ thống điều hướng sang thông báo lỗi tạm dừng dịch vụ</p> <p>[4a] Lỗi đường truyền tới MongoDB ngắt quãng → Hệ thống hiển thị cảnh báo bảo trì máy chủ tạm thời</p>                                                                                                                                                                                                                                                                                                                                                                                                    |

**Bảng 2.5:** Đặc tả use case Duyệt sản phẩm động (UC-07)

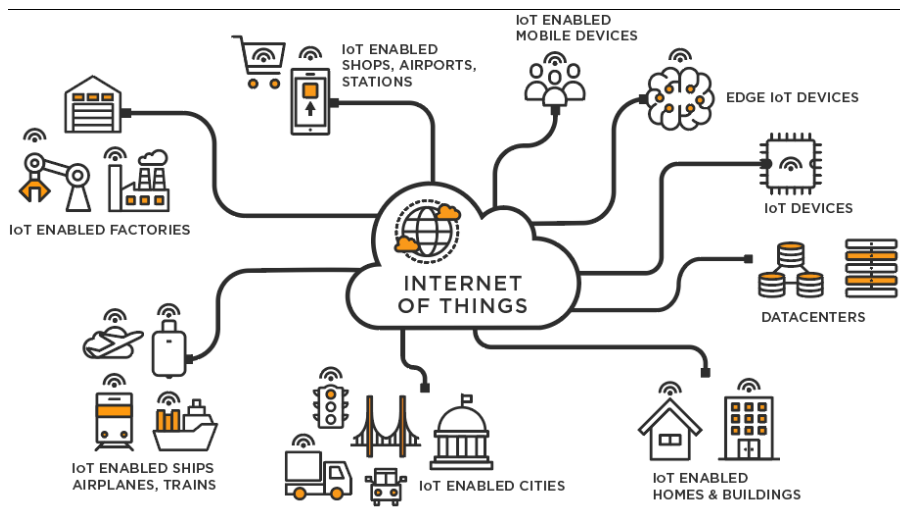
Bảng 2.5 tập trung biểu diễn quy trình xử lý luồng sự kiện cốt lõi định hình kiến trúc Zero-file khi hệ thống phản ứng linh động thông qua các tập tin siêu văn bản JSON theo thời gian thực.



**2.3.5 Đặc tả use case: Thanh toán và Đơn hàng (UC-09)**

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tên use case</b>    | Thanh toán và đơn hàng                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Tác nhân</b>        | End Customer                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Mục đích</b>        | Hoàn thành tiến trình mua sắm và ghi nhận giao dịch dòng tiền vào hệ thống                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Tiền điều kiện</b>  | End Customer đã lựa chọn tối thiểu một món đồ vào khoang chứa mua hàng                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Hậu điều kiện</b>   | Đơn hàng được cập nhật trạng thái rõ ràng và lệnh thông báo qua email được vận hành                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Luồng chính</b>     | <p>1. Khách hàng bấm lệnh thanh toán, chốt thông tin địa chỉ giao hàng và phương thức giao dịch</p> <p>2. Mặt tiền gọi hàm POST /api/v1/orders lên hệ thống trung tâm quản trị cơ sở dữ liệu</p> <p>3. Hệ thống tạo nháp đơn hàng cùng mức thanh toán tạm thời chờ kết quả</p> <p>4. Quá trình kết nối cổng thanh toán kích hoạt, mã giao dịch được lưu vết lại</p> <p>5. Cổng Gateway gửi hàm callback webhook xác nhận tiền đã chảy vào tài khoản</p> <p>6. Hệ thống lỗi đối soát verify signature của webhook, xác thực an toàn tuyệt đối</p> <p>7. Lệnh cập nhật trạng thái update order status được tiến hành, kéo theo gửi email xác nhận và bảng dashboard nháy tín hiệu đơn mới</p> |
| <b>Luồng phát sinh</b> | <p>[5a] Khách hủy thanh toán giữa chừng → Hủy lệnh nháp đơn hàng, giải phóng lượng kho dự phòng</p> <p>[6a] Mã chứng thực chữ ký webhook không khớp chuẩn → Kích hoạt báo động xâm nhập và từ chối nâng trạng thái đơn</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

**Bảng 2.6:** Đặc tả use case Thanh toán và Đơn hàng (UC-09)



**Hình 2.7:** Biểu đồ hoạt động quy trình thanh toán và xác nhận đơn hàng

Hình 2.7 xác nhận chặt chẽ cơ chế đóng kín dữ liệu thanh toán với bên thứ ba một cách an toàn.

## 2.4 Yêu cầu phi chức năng

Hệ thống cần đáp ứng thời gian phản hồi dưới 500ms cho cold cache và dưới 10ms cho warm cache của hệ thống Redis. Mục tiêu vận hành đồng thời 20.000 shop trên hạ tầng cấu hình khiêm tốn 16GB RAM đặt ra yêu cầu vô cùng nghiêm ngặt về tối ưu bộ nhớ. Cụ thể, NestJS phải bị giới hạn cỡ cấu trúc cấp phát vùng nhớ tối đa 1024MB, cả hai cụm cơ sở dữ liệu PostgreSQL và MongoDB phân bổ mỗi thành phần tối đa 512MB RAM, còn đối với máy chủ truy xuất đệm Redis được thiết lập vòng lặp xử lý không vượt quá 256MB. Xét về độ tin cậy, hệ thống cam kết duy trì chuẩn hoạt động ở mức uptime đạt từ 99.5% trở lên, đồng thời không cộng dồn thời gian đóng cổng để cấu hình bảo trì. Một cơ chế quy trình tự động được thiết kế nhằm sao lưu chạy mỗi ngày thông qua các tiến trình Kubernetes CronJob khép kín. Mọi lỗi bất ngờ phát sinh từ cơ sở dữ liệu đều được quản lý chặt chẽ qua cơ chế ném ngoại lệ `InternalServerErrorException` của NestJS, tuyệt đối không để lộ mã ngăn xếp truy vết ra thông báo trả về cho máy khách. Đi sâu hơn về khía cạnh tính dễ dùng, toàn bộ trung tâm điều khiển Admin Dashboard cần hỗ trợ tối ưu cho các người dùng cuối không có kiến thức kỹ thuật nâng cao. Giao diện thiết kế phải tuân thủ nghiêm chỉnh những quy định thiết kế tối giản, và mọi tính năng tác vụ trọng yếu như đăng hình ảnh gốc, thay màu nền thương hiệu phải dễ dàng vận hành với tổng thời gian thao tác tối đa không vượt qua thời lượng 3 phút. Phân tích đến yếu tố bảo mật, quá trình cấp quyền chứng thực xác thực thông qua cấu trúc JWT chuẩn mã hóa HS256. Mọi đường dẫn truy cập API đều yêu cầu khắt khe thông số xác minh Authorization header ngay ở những truy vấn đơn giản nhất. Đặc biệt, bản

chất môi trường thiết lập đa khách thuê đặt nền móng chắc chắn đảm bảo cô lập dữ liệu tuyệt đối; chủ cửa hàng này sẽ hoàn toàn bị chặn đứng quyền truy xuất dữ liệu từ bất cứ cửa hàng nào khác trong cùng máy chủ. Secret mã hóa cực đoan được nạp qua cấu hình biến môi trường và không bao giờ xuất hiện lộ liễu trên các mã nguồn vật lý, đồng thời mã hóa TLS 1.3 bảo trợ toàn vẹn luồng dữ liệu liên thông. Đứng ở các góc độ yêu cầu kỹ thuật nền tảng, hệ thống cấu hình chạy tốt trên máy chủ có nhân lõi Node.js phiên bản 22 trở lên. Hệ thống phía máy khách hỗ trợ ổn định cho trình duyệt Chrome phiên bản 100, Firefox bản 100 hay Safari từ bản 15. Dữ liệu chạy dựa trên quy chuẩn cấu trúc PostgreSQL 16 tích hợp ORM Prisma và MongoDB 7 đi kèm Mongoose linh động. Hạ tầng triển khai kết dính toàn cục thông qua Docker Compose trong khi vận hành phát triển và nâng cấp thành hệ quản trị Kubernetes phân luồng tại máy chủ sản xuất, tất cả đồng loạt được thiết kế và quản trị theo cấu trúc tự động Turborepo nhằm gia tăng sức bật thời gian đóng gói biên dịch hệ thống.

Chương này đã làm rõ hiện trạng thị trường khi khảo sát ba nguồn phân tích cụ thể, từ đó hệ thống đã định hướng và chỉ điểm rõ 10 nhóm tính năng cốt lõi. Từ 10 chức năng quan trọng này, chương cũng đã đặc tả cẩn thận từng sự kiện diễn ra của 5 trường hợp sử dụng ưu tú nhất, bao gồm luôn những yêu cầu phi chức năng cần đáp ứng ở bước chạy đà thực tế. Chương tiếp theo trình bày các công nghệ được lựa chọn để hiện thực hóa các yêu cầu đã xác định.

## CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định các yêu cầu chức năng và phi chức năng của hệ thống ShopVolo v2 từ quá trình khảo sát kỹ lưỡng thực tiễn người dùng. Chương 3 này sẽ trình bày các công nghệ được lựa chọn để đáp ứng những yêu cầu đó, đồng thời phân tích rõ lý do tại sao chúng được tin dùng hơn so với các giải pháp thay thế trên thị trường. Về mặt tổng thể, nội dung sẽ được liệt kê sơ bộ thông qua các nhóm công nghệ lõi như kiến trúc quản lý tập trung mã nguồn, hệ thống lập trình máy chủ phụ trợ, nền tảng phát triển giao diện hiển thị, cấu trúc cơ sở dữ liệu chuyên biệt, và cuối cùng là các công nghệ đóng gói hạ tầng cũng như lưu trữ đối tượng.

### 3.1 Turborepo — Quản lý Monorepo

Để đáp ứng yêu cầu phi chức năng tại 2.4 về tính bảo trì và quy trình xây dựng tự động hóa, hệ thống yêu cầu quản lý 4 ứng dụng (admin, api-core, storefront, cli-tool) và 5 gói phần mềm dùng chung (ui-library, schema, database, i18n, master-templates) một cách nhất quán trong cùng một kho lưu trữ tập trung.

Các công cụ quản lý kho chứa mã nguồn tập trung phổ biến hiện nay bao gồm (i) Nx, (ii) Lerna, và (iii) trình quản lý không gian làm việc pnpm thuần túy.

Turborepo được chọn vì (i) khả năng lưu trữ bộ nhớ đệm dạng tăng dần giúp hệ thống chỉ tiến hành biên dịch lại đúng các gói phần mềm thực sự bị thay đổi, giúp giảm triệt để thời gian chạy các luồng tự động, (ii) công cụ này thiết lập được quy trình biên dịch tự động dựa vào khái niệm đồ thị có hướng phi chu trình, và (iii) không yêu cầu quá trình chuyển đổi cấu trúc phức tạp khi tích hợp sẵn với không gian làm việc của npm đã tồn tại [8].

### 3.2 NestJS — Framework Backend

Để đáp ứng các chức năng được thiết kế chi tiết tại 2.3, hệ thống yêu cầu một giải pháp xây dựng giao diện lập trình ứng dụng có cấu trúc mô-đun rõ ràng như quản lý cửa hàng, sản phẩm, đơn hàng, đồng thời hỗ trợ tiêm phụ thuộc, xây dựng tầng trung gian phục vụ đa thuê bao và hỗ trợ định dạng tĩnh cấu trúc mạnh mẽ.

Các công cụ lập trình phụ trợ bằng ngôn ngữ JavaScript nổi bật khác bao gồm (i) Express.js, (ii) Fastify, và (iii) Hono.

NestJS được chọn vì (i) kiến trúc mô-đun mô phỏng hệ thống Angular hỗ trợ tổ chức mã nguồn theo từng cụm giới hạn rõ ràng, (ii) tích hợp rất tốt với bộ nhớ ngữ cảnh bất đồng bộ nhằm truyền tải định danh khách thuê một cách tự động, và (iii) sử dụng trình trang trí kết hợp với cơ chế lọc ngoại lệ giúp xử lý thống nhất các lỗi nội bộ trên toàn bộ máy chủ [9].

Cụ thể, tầng trung gian tự động giải mã thông tin định danh khách thuê từ phần mào đầu truy vấn, sau đó lưu vào bộ nhớ bất đồng bộ để toàn bộ quá trình đọc ghi cơ sở dữ liệu tiếp theo đều tự động gắn kết.

### 3.3 Next.js — Framework Frontend

Để đáp ứng yêu cầu hiệu năng tại 2.4, hệ thống đòi hỏi nền tảng mặt tiền phải phản hồi ở tốc độ dưới mức 500 mili-giây, cùng với đó trang quản trị của người bán phải tải được các dạng biểu mẫu cấu trúc phức tạp với độ tương tác phản ứng tức thì.

Các nền tảng xây dựng giao diện nổi bật thay thế bao gồm (i) bộ đôi React và Vite, (ii) Remix, và (iii) hệ thống Nuxt.js dựa trên Vue.

Next.js phiên bản 15 được chọn vì (i) bộ định tuyến ứng dụng hỗ trợ tính năng kết xuất các phần tử phía máy chủ giúp giảm thiểu dung lượng mã tải về cho khách truy cập, (ii) các hàm trung gian hoạt động ở biên cho phép nhận diện tên miền khách thuê và định tuyến từ rất sớm, và (iii) cơ chế tái tạo tĩnh tăng dần cung cấp khả năng tự động thiết lập thời gian sống cho bộ nhớ đệm của các trang mô tả sản phẩm một cách cực kỳ linh hoạt [10].

### 3.4 PostgreSQL và Prisma ORM — Cơ sở dữ liệu quan hệ

Để đáp ứng việc quản lý các thực thể có cấu trúc định hình cố định đã được xác định tại 2.1 như chủ cửa hàng, tài khoản, đơn vị mua hàng, hệ thống đòi hỏi một giải pháp bảo đảm tính nguyên tử của giao dịch, ràng buộc chặt chẽ khóa ngoại và khả năng trích xuất thông tin bảng thống kê nâng cao.

Các giải pháp quản lý bảng biểu cấu trúc thay thế bao gồm (i) MySQL 8, (ii) SQLite ở môi trường cục bộ, và (iii) hệ thống phân tán CockroachDB.

PostgreSQL 16 kết hợp công cụ Prisma được chọn vì (i) có khả năng hỗ trợ kiểu định dạng JSONB nhằm lưu vết các tham số động cực tốt khi cần, (ii) cung cấp chức năng bảo mật truy xuất cấp độ hàng dữ liệu hỗ trợ mạnh mẽ cho định hướng thiết kế đa thuê bao, và (iii) công cụ Prisma ORM tự động khởi tạo hệ sinh thái kiểu tĩnh và phiên bản hóa các đợt di chuyển cấu trúc mạnh mẽ [11], [12].

### 3.5 MongoDB và Mongoose — Cơ sở dữ liệu tài liệu

Để đáp ứng nền tảng kiến trúc Zero-file đã vạch ra tại 1.3, việc lưu trữ các tệp cấu hình JSON phải giải quyết bài toán lồng nhau sâu sắc của bộ khung nền và bộ tùy chỉnh cửa hàng, đồng thời hỗ trợ cập nhật mở rộng không cần theo định dạng cố định khắt khe của kho lưu trữ quan hệ truyền thống.

Các nền tảng giải quyết dữ liệu văn bản phi cấu trúc khác gồm (i) CouchDB, (ii) giải pháp đám mây DynamoDB, và (iii) lợi dụng kiểu JSONB của chính cơ sở dữ

liệu PostgreSQL.

MongoDB phiên bản 7 kết hợp bộ thư viện Mongoose được chọn vì (i) mô hình dữ liệu tài liệu cực kỳ tự nhiên khi diễn dịch các khối phần tử giao diện lồng ghép, (ii) tính năng thẩm định dữ liệu linh động của Mongoose cho phép kiểm soát chất lượng tham số cốt lõi mà vẫn chừa lại không gian để linh hoạt biến hóa, và (iii) thuật toán kết hợp đệ quy sâu dễ dàng tích hợp để đồng bộ các mảng dữ liệu đồ sộ [13].

### 3.6 Redis — Cache In-memory

Để đáp ứng yêu cầu hiệu năng cực kỳ khắt khe tại 2.4, hệ thống đặt mục tiêu phản hồi trong 10 mili-giây khi duyệt trang phục vụ cho việc lấy cấu hình ở phần 2.3.4, điều hoàn toàn bất khả thi nếu cứ liên tục gọi cấu trúc dữ liệu thô.

Các phương pháp thay thế giải quyết độ trễ phổ biến bao gồm (i) Memcached, (ii) kiểm soát bộ nhớ qua giao thức HTTP cơ bản, và (iii) sử dụng mạng phân phối nội dung thuần túy.

Redis được chọn vì (i) hệ lưu trữ trên vùng nhớ trực tiếp đảm bảo độ trễ ở mức vô cùng hoàn hảo, (ii) khả năng thiết lập tuổi thọ tự động hết hạn mà không cần tốn thêm tài nguyên máy chủ để tự thu dọn rác, và (iii) giao thức phát tin hiệu độc lập rất hữu ích cho thao tác vô hiệu hóa bộ đệm khi cửa hàng thực hiện thao tác cập nhật giao diện hiển thị [14].

### 3.7 MinIO — Object Storage

Để đáp ứng quy trình quản lý thông tin ở phần 2.3.3, hệ thống yêu cầu một giải pháp lưu trữ tệp tin hình ảnh sản phẩm định dạng nhị phân theo nhiều biến thể kích cỡ, đây là một môi trường dữ liệu không thể dồn nén trực tiếp vào kho lưu trữ bảng biểu truyền thống.

Các dịch vụ hỗ trợ cung cấp kho chứa đối tượng thay thế thường được tiếp cận là (i) hệ sinh thái đám mây AWS S3, (ii) không gian lưu trữ Cloudflare R2, và (iii) ghi tệp thẳng vào hệ thống lưu trữ tĩnh trên máy chủ nội bộ.

MinIO được chọn vì (i) sở hữu chuẩn giao thức giao tiếp y hệt như cấu trúc S3, giúp việc di dời toàn bộ dữ liệu hệ thống lên nền tảng trực tuyến lớn không gặp trở ngại gì sau này, (ii) bản thân phần mềm cung cấp khả năng lưu trữ vận hành ngay trên hạ tầng đóng gói khép kín giúp tiết kiệm kinh phí tối đa, và (iii) khả năng ủy quyền trực tiếp liên kết giúp trang mặt tiền có thể đưa dữ liệu lên máy chủ ảnh bỏ qua khâu xử lý luân chuyển ở trung tâm [15].

### 3.8 Better Auth — Xác thực và Phân quyền

Để đáp ứng đầy đủ yêu cầu bảo mật ở 2.4, toàn hệ thống phụ thuộc vào cơ chế kiểm soát truy cập phân tầng cho quản trị viên và chủ cửa hàng theo định dạng thẻ thông hành JWT, kèm theo thông số định danh thiết lập đa thuê bao.

Các công cụ lập trình xác thực bên thứ ba khả dĩ cho giải pháp này bao gồm (i) thư viện chuẩn NextAuth.js, (ii) sử dụng tự động Passport.js kết hợp thẻ xác thực, và (iii) hệ thống dịch vụ Clerk.

Thư viện Better Auth được chọn vì (i) xây dựng bằng ngôn ngữ tĩnh TypeScript bảo vệ hệ thống tuyệt đối ở cấp độ biên dịch, (ii) khả năng liên kết tự nhiên ngay trên kiến trúc định tuyến nâng cao của ứng dụng mặt tiền, và (iii) hỗ trợ kết nối thẳng vào trình điều khiển Prisma để đồng bộ phiên làm việc của người dùng vào hệ thống lưu trữ truyền thống [16].

### 3.9 Docker và Kubernetes — Containerization và Orchestration

Để đáp ứng yêu cầu phi chức năng về độ tin cậy được thể hiện tại 2.4, tất cả hệ quản trị phụ trợ khổng lồ phải được giữ cho vận hành đồng bộ và ổn định ở mọi phiên bản triển khai, chưa kể đến yêu cầu bắt buộc phải sao lưu tự động định kỳ vào mỗi ngày cuối ngày.

Các phương pháp triển khai quản lý cụm máy chủ truyền thống có thể kể đến như (i) cấu hình trực tiếp thủ công bằng tiện ích hệ điều hành, (ii) trình quản lý Docker Swarm tập trung, và (iii) hệ thống phân phối của Nomad.

Sự kết hợp giữa Docker và Kubernetes được chọn vì trong giai đoạn phát triển, hệ thống tự chứa cấp cho tốc độ khởi chạy tối đa, còn khi vận hành sản xuất thì (i) hệ sinh thái CronJob điều phối chuẩn xác các tác vụ dọn dẹp và dự phòng theo kế hoạch, (ii) chiến lược thay thế cuộn chiếu cho phép cập nhật phiên bản diễn ra hoàn toàn không làm ngưng trệ bất cứ tương tác nào, và (iii) trình quản lý bí mật vô hiệu hóa hoàn toàn mọi rủi ro lộ lọt cấu hình riêng tư [17], [18].

| Công nghệ    | Phiên bản | Loại               | Mục đích trong đề tài                          |
|--------------|-----------|--------------------|------------------------------------------------|
| Turborepo    | 2.x       | Build Tool         | Quản lý monorepo, incremental build cache      |
| NestJS       | 11.x      | Backend Framework  | REST API, multi-tenancy, DI container          |
| Next.js      | 15.x      | Frontend Framework | Admin Dashboard + Storefront Engine (SSR)      |
| PostgreSQL   | 16        | RDBMS              | Lưu Tenant, User, Order, Product (structured)  |
| Prisma       | 5.x       | ORM                | Type-safe query + schema migration             |
| MongoDB      | 7         | Document DB        | Lưu Layout, Config, Navigation (flexible JSON) |
| Mongoose     | 8.x       | ODM                | Schema validation cho MongoDB                  |
| Redis        | 7.x       | In-memory Cache    | Cache storefront config (TTL=1h)               |
| MinIO        | Latest    | Object Storage     | Lưu ảnh sản phẩm, S3-compatible                |
| Better Auth  | 1.x       | Auth Library       | JWT, session, role-based access                |
| Docker       | 26.x      | Containerization   | Môi trường development nhất quán               |
| Kubernetes   | 1.29+     | Orchestration      | Production deployment, CronJob                 |
| Tailwind CSS | 4.x       | CSS Framework      | Styling Admin Dashboard + Storefront           |
| Zod          | 3.x       | Validation         | Schema validation API request/response         |

**Bảng 3.1:** Tổng hợp công nghệ sử dụng trong hệ thống ShopVolo v2

Chương này đã làm rõ tất cả các công nghệ được chọn để đáp ứng tương ứng từng yêu cầu cốt lõi từ Chương 2. Nhấn mạnh sự bổ sung linh hoạt và hợp lý giữa PostgreSQL trong quản trị các cấu trúc dữ liệu cứng nhắc và MongoDB trong xử lý thông tin mềm dẻo đa biến, cũng như sự đồng hành giữa Docker Compose lúc phát triển và Kubernetes ở giai đoạn vận hành thực tế. Trên cơ sở các công nghệ đã lựa chọn, Chương 4 trình bày thiết kế kiến trúc và kết quả triển khai hệ thống.



## CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

### 4.1 Thiết kế kiến trúc

#### 4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được thiết kế theo kiến trúc **Monorepo** (Turborepo) kết hợp với tư tưởng **Modular Monolith** cho tầng backend. Kiến trúc Monorepo cho phép quản lý tập trung toàn bộ mã nguồn frontend, backend và các gói dùng chung trong một kho lưu trữ duy nhất, tối ưu hóa việc chia sẻ kiểu dữ liệu (type-safety) và quy trình CI/CD.

Hệ thống bao gồm ba thành phần chính:

- **Frontend:** Hai ứng dụng Next.js độc lập – *storefront* (khách hàng mua sắm) và *admin* (chủ cửa hàng quản lý).
- **Backend (*api-core*):** NestJS được tổ chức theo các module nghiệp vụ (Auth, Shop, Order, Catalog, Layout, Inventory, Payment, Media, v.v.) với khả năng mở rộng thành microservices.
- **Shared Packages:** Năm gói thư viện nội bộ: database (Prisma ORM), schema (Zod validation), master-templates, ui-registry, và *utils*.

Đối với bài toán **Đa hệ thuê (Multi-tenancy)**, hệ thống áp dụng chiến lược *Shared Database, Row-Level Isolation*: mọi thực thể nghiệp vụ đều mang khóa ngoại `shopId`; phân lập logic được thực thi qua `AsyncLocalStorage` ở tầng backend, đảm bảo mọi truy vấn tới database đều được tự động gắn đúng ngữ cảnh cửa hàng mà không cần truyền tham số qua từng lớp.

#### 4.1.2 Thiết kế tổng quan

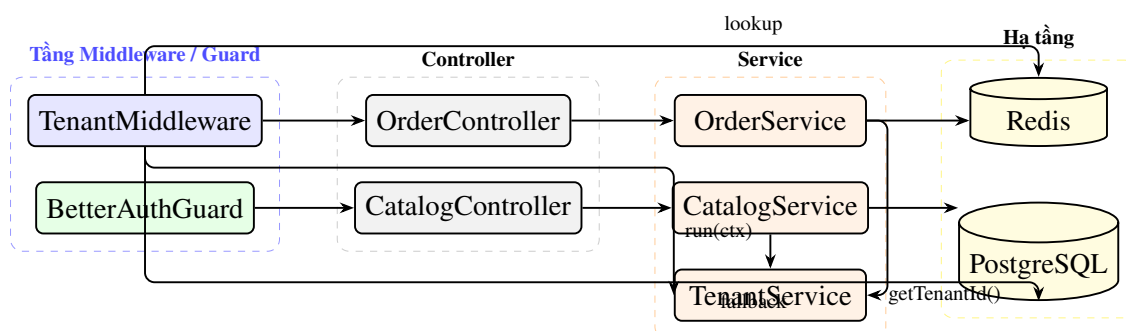
Biểu đồ gói dưới đây mô tả kiến trúc phân tầng của toàn bộ hệ thống. Tầng ứng dụng (Applications) phụ thuộc vào tầng gói dùng chung (Packages), và cả hai cùng phụ thuộc vào tầng hạ tầng (Infrastructure). Không có sự phụ thuộc ngược chiều (từ tầng dưới lên tầng trên) nhằm đảm bảo nguyên tắc thiết kế hướng phụ thuộc một chiều (Dependency Direction).

Mô tả vai trò từng thành phần:

- **storefront & admin:** Hai ứng dụng Next.js 16 độc lập, sử dụng chung `ui-registry` (component library) và `i18n` (đa ngôn ngữ). Giao tiếp với `api-core` qua REST API.
- **api-core:** Ứng dụng NestJS đóng vai trò backend duy nhất, phụ thuộc vào database (Prisma/PostgreSQL), schema (Zod DTO), và master-templates.
- **database:** Gói bao gồm Prisma schema và PrismaService. Dữ liệu quan hệ lưu trên PostgreSQL.
- **schema:** Gói định nghĩa kiểu dữ liệu dùng chung (Zod) giữa frontend và backend.

### 4.1.3 Thiết kế chi tiết gói

Bên trong `api-core`, kiến trúc được phân tầng theo mô hình *Middleware* → *Guard* → *Controller* → *Service* → *Repository*. Cơ chế Multi-tenancy được hiện thực hóa theo luồng sau:



**Hình 4.1:** Thiết kế chi tiết gói trong `api-core`: Luồng xử lý đa hệ thuê

TenantMiddleware được đăng ký toàn cục trong AppModule và áp dụng cho tất cả các route. Nó trích xuất subdomain/header x-tenant-id, kiểm tra Redis (TTL 300s), nếu cache miss thì truy vấn PostgreSQL, sau đó gọi TenantService.run(ctx, next) để lưu ngữ cảnh vào AsyncLocalStorage. Mọi service nghiệp vụ (OrderService, CatalogService, v.v.) gọi TenantService.getTenantId() để lấy shopId tự động mà không cần truyền tham số.

### 4.2 Thiết kế chi tiết

#### 4.2.1 Thiết kế giao diện

Giao diện hệ thống được thiết kế theo chuẩn Component-based, sử dụng TailwindCSS v4 và gói ui-registry tập trung tất cả các component tái sử dụng. Các quy chuẩn thiết kế giao diện:

- **Tính đáp ứng (Responsive):** Hỗ trợ 3 breakpoint: Mobile (< 768px), Tablet (768–1024px), Desktop (> 1024px).
- **Giao diện động (Dynamic Layout):** Storefront không có giao diện cố định mà render động từ JSON Schema trả về bởi LayoutService, cho phép chủ cửa hàng tùy biến trang web qua kéo thả.
- **Phản hồi người dùng:** Mọi hành động đều có trạng thái loading, thông báo thành công/lỗi rõ ràng.

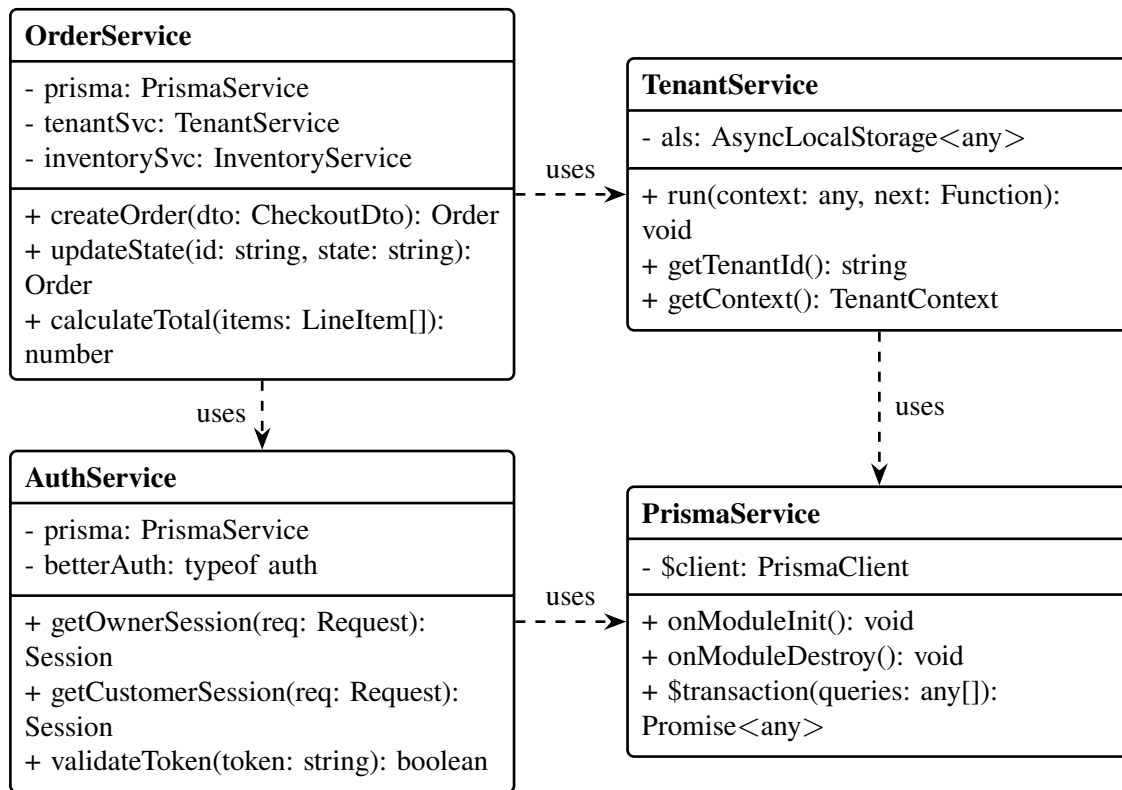
(Hình ảnh giao diện thực tế của dự án sẽ được chèn vào đây sau khi hoàn thiện UI)

#### 4.2.2 Thiết kế lớp

Phần này trình bày thiết kế chi tiết các thuộc tính và phương thức cho các lớp dịch vụ (Service) cốt lõi của hệ thống, cùng với luồng tương tác giữa chúng qua biểu đồ trình tự cho các use case quan trọng.

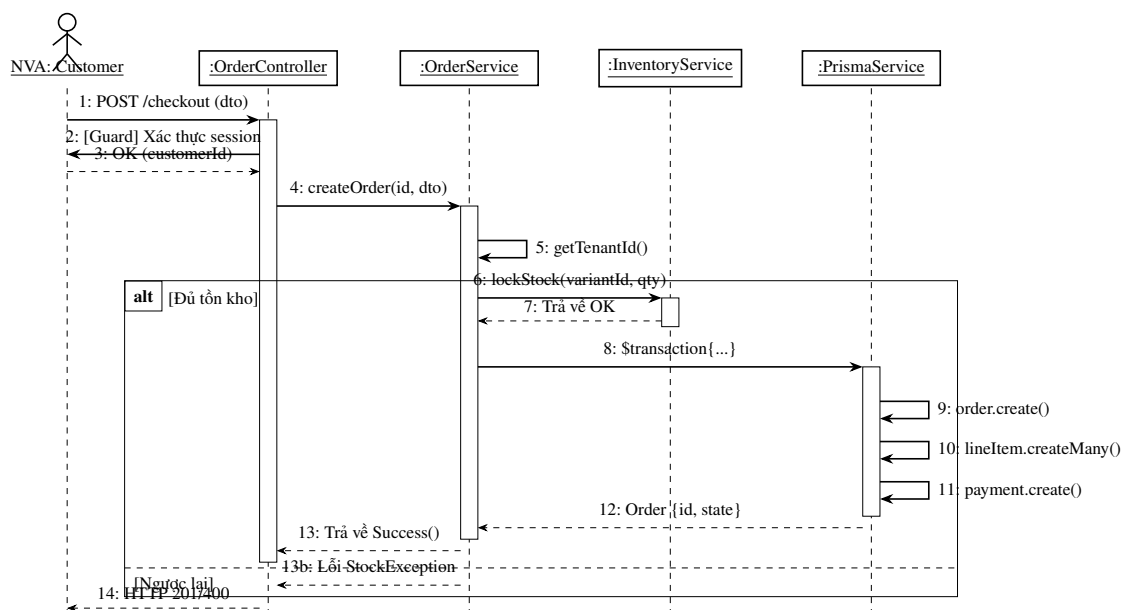
##### a, Thiết kế chi tiết lớp (Class Diagram)

Dưới đây là biểu đồ thiết kế lớp cho 4 dịch vụ chủ đạo nhất trong api-core: OrderService, TenantService, AuthService và PrismaService.



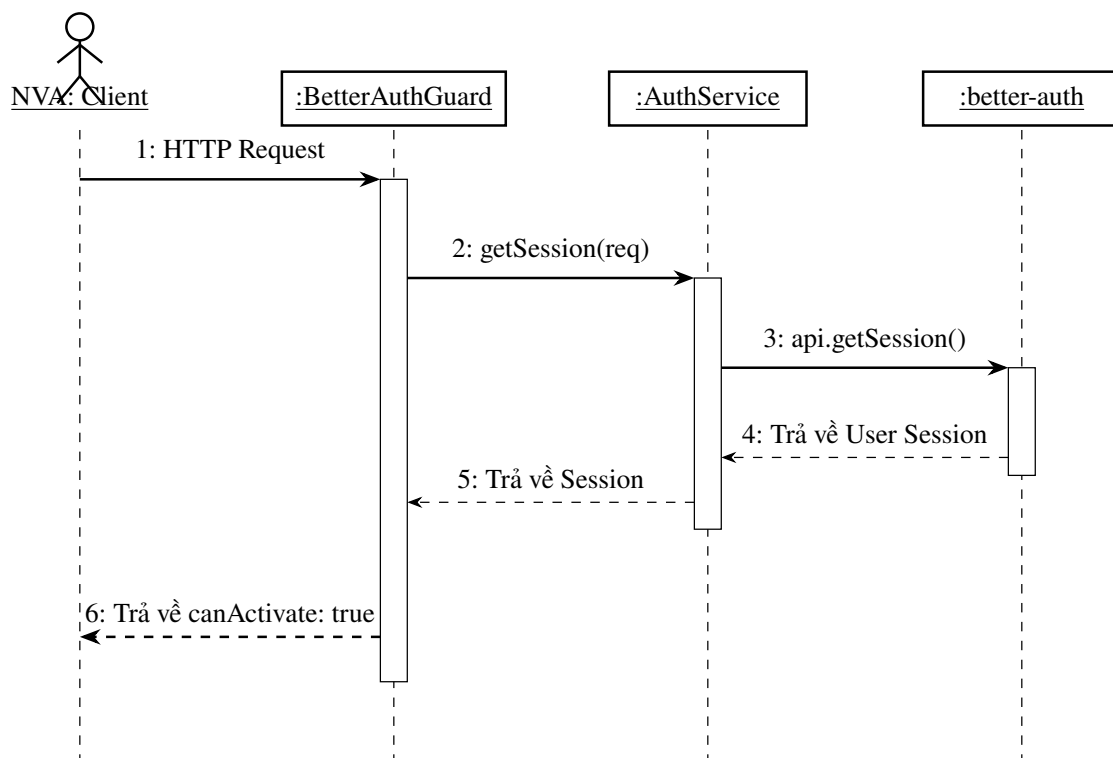
Hình 4.2: Biểu đồ thiết kế chi tiết các lớp cốt lõi

## b, Luồng 1: Khách hàng đặt hàng (Checkout)



Hình 4.3: Biểu đồ trình tự: Luồng Checkout (Đặt hàng)

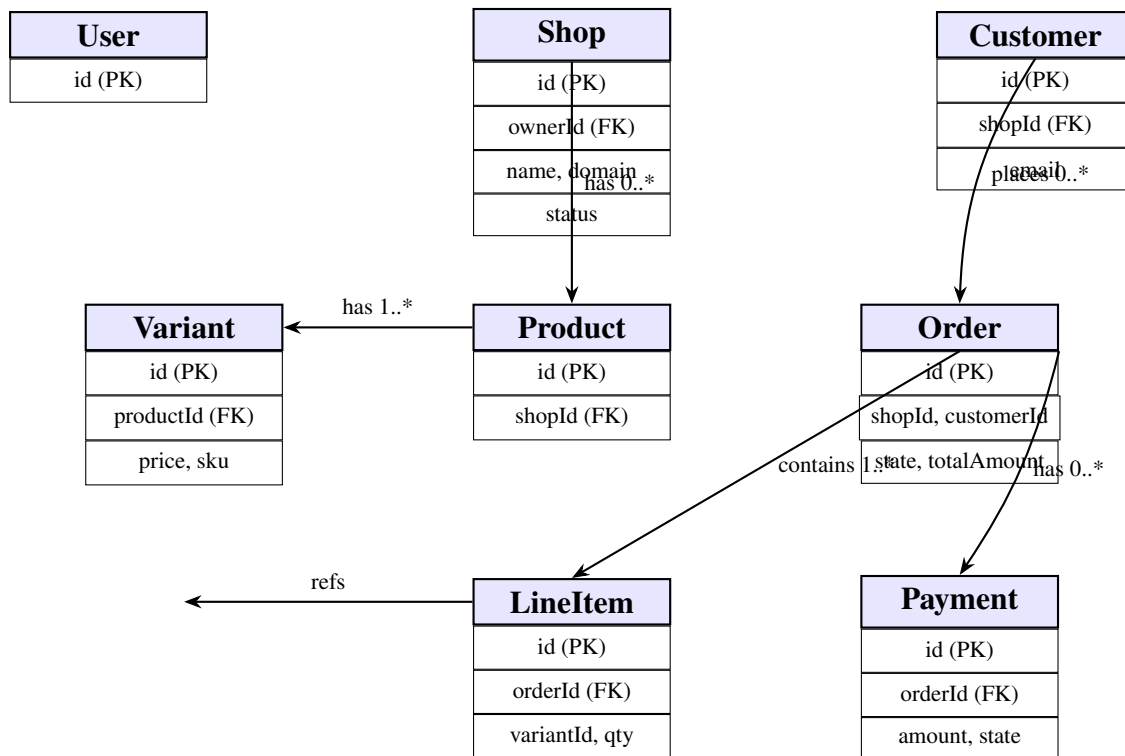
**c, Lồng 2: Xác thực và phân quyền (BetterAuthGuard)**



**Hình 4.4:** Biểu đồ trình tự: Lồng xác thực BetterAuthGuard

### 4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng **PostgreSQL** làm CSDL quan hệ chính, quản lý qua **Prisma ORM v5.22.0**. Sơ đồ thực thể liên kết (E-R Diagram) cho các thực thể cốt lõi được thể hiện bên dưới.



**Hình 4.5:** Sơ đồ E-R các thực thể cốt lõi của hệ thống (PostgreSQL/Prisma)

Ngoài PostgreSQL, hệ thống sử dụng thêm:

- **MongoDB (Mongoose):** Lưu trữ schema bố cục giao diện (GlobalLayout, PageLayout) dưới dạng JSON linh hoạt, phù hợp với tính động của Layout Engine.
- **Redis:** Lưu cache kết quả định danh tenant (TTL: 300 giây), giảm tải truy vấn PostgreSQL trong các request có tần suất cao.
- **MinIO:** Object Storage tương thích S3 để lưu trữ file media (ảnh sản phẩm). Tích hợp qua AWS SDK v3 với `forcePathStyle: true`.

### 4.3 Xây dựng ứng dụng

#### 4.3.1 Thư viện và công cụ sử dụng

| Mục đích           | Công cụ / Thư viện   | Phiên bản |
|--------------------|----------------------|-----------|
| Ngôn ngữ lập trình | TypeScript           | v5.7.3    |
| Frontend Framework | Next.js (App Router) | v16.2.6   |
| UI Library         | React                | v19.2.3   |
| CSS Framework      | TailwindCSS          | v4        |
| Backend Framework  | NestJS               | v11.0.1   |
| Database ORM       | Prisma ORM           | v5.22.0   |
| Xác thực           | Better Auth          | v1.6.2    |
| Cache              | Redis (ioredis)      | v5.11.0   |
| Message Queue      | Bull (BullMQ)        | v11.0.4   |
| Object Storage     | MinIO / AWS SDK v3   | v3.1057.0 |
| Schema Validation  | Zod                  | v3.24.3   |
| NoSQL ODM          | Mongoose             | v8.2.0    |
| Build Tool         | Turborepo            | monorepo  |
| IDE                | VS Code              | —         |

**Bảng 4.1:** Danh sách thư viện và công cụ sử dụng trong dự án

#### 4.3.2 Kết quả đạt được

Hệ thống hoàn thành việc xây dựng kiến trúc nền tảng SaaS thương mại điện tử Đa hệ thuê với đầy đủ các module cốt lõi.

| Thông số                         | Giá trị       |
|----------------------------------|---------------|
| Tổng số tệp mã nguồn (TS/TSX)    | 263 tệp       |
| Tổng số dòng mã nghiệp vụ        | 20,316 dòng   |
| Số module Service trong api-core | 21 service    |
| Số Controller API trong api-core | 16 controller |
| Shared Packages nội bộ           | 5 gói         |
| Số bảng CSDL (PostgreSQL)        | 30 model      |

**Bảng 4.2:** Thống kê thông số kỹ thuật dự án

#### 4.3.3 Minh họa các chức năng chính

(Ảnh chụp màn hình thực tế của các chức năng sau đây sẽ được bổ sung:)

1. Luồng đăng ký và khởi tạo cửa hàng mới (Onboarding)
2. Quản lý sản phẩm và tồn kho trên Admin Dashboard
3. Kéo thả tùy biến bố cục trang Storefront (Layout Builder)
4. Luồng mua hàng và thanh toán của khách hàng

## 5. Theo dõi và cập nhật trạng thái đơn hàng

### 4.4 Kiểm thử

Hệ thống sử dụng **Jest** kết hợp `@nestjs/testing` để thực hiện Unit Test và Integration Test cho tầng backend. Kỹ thuật được áp dụng:

- **Dependency Injection Mocking:** Toàn bộ `PrismaService`, `TenantService` và `Redis Cache` được mock thay thế bằng đối tượng giả nhằm đảm bảo test chạy độc lập với hạ tầng.
- **Unit Test:** Tập trung vào logic nghiệp vụ phức tạp: `AuthService.getOwnerSession()`, xác minh điều kiện giỏ hàng trong `OrderService.createOrder()`.

Các trường hợp kiểm thử cho chức năng **Xác thực (Auth)**:

| TC | Mô tả                            | Đầu vào             | Kết quả mong đợi        |
|----|----------------------------------|---------------------|-------------------------|
| 1  | Lấy session với cookie hợp lệ    | Cookie hợp lệ       | 200 OK + user object    |
| 2  | Lấy session với cookie hết hạn   | Cookie expired      | null → 401 Unauthorized |
| 3  | Đổi tên hợp lệ                   | newName khác tên cũ | 200 OK + updated:true   |
| 4  | Đổi tên trùng tên cũ             | newName = tên cũ    | 400 Bad Request         |
| 5  | Truy cập endpoint không có Guard | Request             | 200 OK (Public)         |

**Bảng 4.3:** Các trường hợp kiểm thử chức năng Xác thực

### 4.5 Triển khai

Hệ thống được thiết kế hướng Cloud-Native với khả năng đóng gói và triển khai qua Docker. Mỗi ứng dụng trong Monorepo (`storefront`, `admin`, `api-core`) được đóng gói thành Docker image riêng biệt và điều phối qua **Docker Compose** hoặc **Kubernetes** tùy môi trường.

Mô hình triển khai:

- **API Gateway / Reverse Proxy:** NGINX đảm nhận việc điều phối traffic, ánh xạ wildcard domain `*.tenant.com` tới ứng dụng Storefront đúng.
- **Xử lý đa hệ thuê tại tầng Next.js:** `Next.js Middleware (proxy.ts)` thực hiện URL Rewrite, chuyển tiếp `x-tenant-id` header tới `api-core`.
- **Stateful Services:** PostgreSQL và Redis triển khai độc lập với persistent volume. MinIO hoạt động như Object Storage nội bộ.